

Perceptron Model for breast Cancer Classification

1) Initial Setup and Data Loading ~

Import sklearn.datasets

Import numpy as np

Import pandas as pd

import matplotlib.pyplot as plt

sklearn.model_selection import train_test_split

• These lines import necessary python libraries:~

- sklearn.datasets: Used to load standard datasets in this case breast Cancer Dataset.
- numpy as np: package for numerical computation
- pandas as pd: Provide data structure & tools for data analysis & manipulation.
- matplotlib.pyplot as plt ~ A plotting library used for creating visualizations.
- from sklearn.model_selection import train_test_split

2) Data Conversion and Initial Inspection

- data = pd.DataFrame(breast_cancer.data, columns = breast_cancer.feature_name) → convert numerical data into a Pandas DataFrame
- The columns are set to breast_cancer.feature_name of different measurement(features) for each tissue sample.

- `data['class'] = breast_cancer.target`

- A new column named 'class' is added to the dataframe.

- This column contains the target labels (`breast_cancer.target`), which represent whether a tissue is malignant or benign.

- `data.head()`

This display the first 5 rows of the Dataframe.

- `data.tail()`

This display the last 5 rows of the Dataframe

- `data.describe()`

This function generates description statistics of the Dataframe's numeric column. it provide information such as count, mean, standard deviation, IQR etc.

- `data.groupby('class').mean()`

- This group the data by the 'class' column and calculate the mean of all features for each class (malignant and benign)

```
x = data.drop("class", axis = 1);
```

```
y = data["class"]
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y,  
                                                    , test_size = 0.2 , stratify = y , random_state = 1)
```

↳ splits data into testing and training

- `plt.plot(x_test.T, 'x')`
`plt.xticks(rotation='vertical')`
`plt.show()`

- `x_linearised_3_train = x_train['mean area'].map(lambda x: 0 if x < 1000 else 1)`

→ perform general linearization on all features

- `x_linearised_train = x_train.apply(pd.cut, bins=2, labels=[1, 0])`

- `plt.plot(x_test.T, 'x')`
`plt.xticks(rotation='vertical')`
`plt.show()`

→ same for testing data

- `x_linearised_train = x_train.apply(pd.cut, bins=2, labels=[1, 0])`

→ convert Dataframe into numpy Array

`x_linearised_test = x_binarised_test.values`

`x_linearised_train = x_binarised_train.values`

- `type(x_binarised_train)`

from sklearn.metrics import accuracy_score.

x_train = X_train.values

x_test = X_test.values

Statistics for Machine learning

divided into two parts:-

1) Descriptive Statistics

- ↳ Central tendency
- ↳ Dispersion method
- ↳ Probability theory
- ↳ Probability distribution Function

2) Inferential Statistics

- ↳ Sample/Population
- ↳ hypothesis testing
- ↳ Z test
- ↳ T test
- ↳ f test
- ↳ alpha test
- ↳ Chi Square test

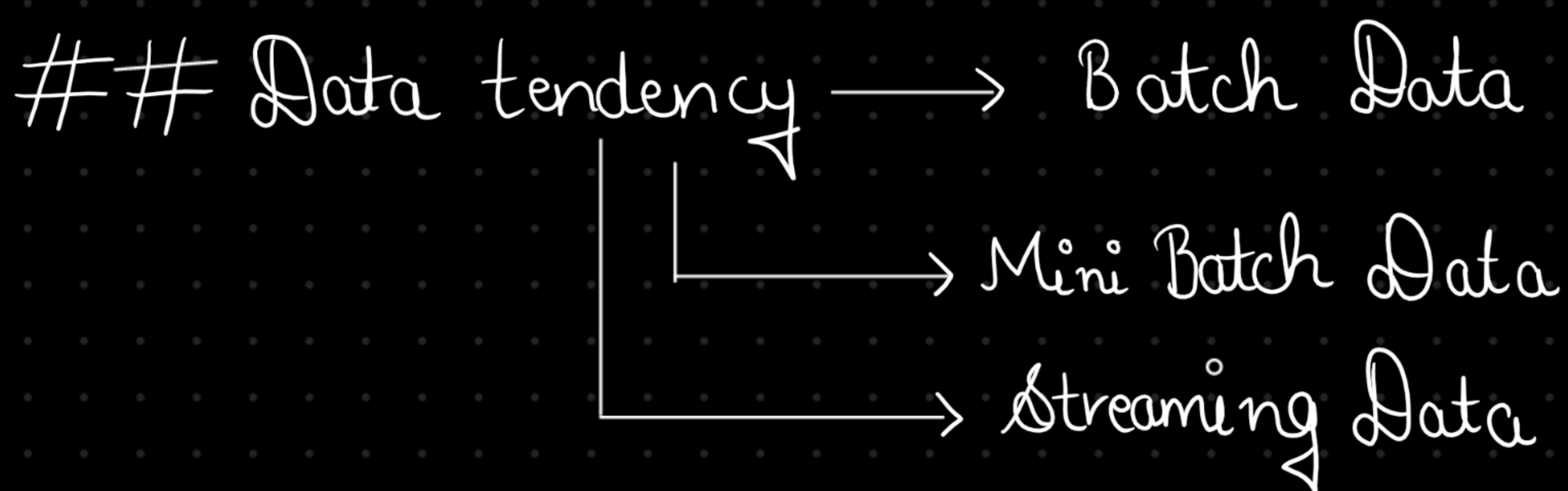
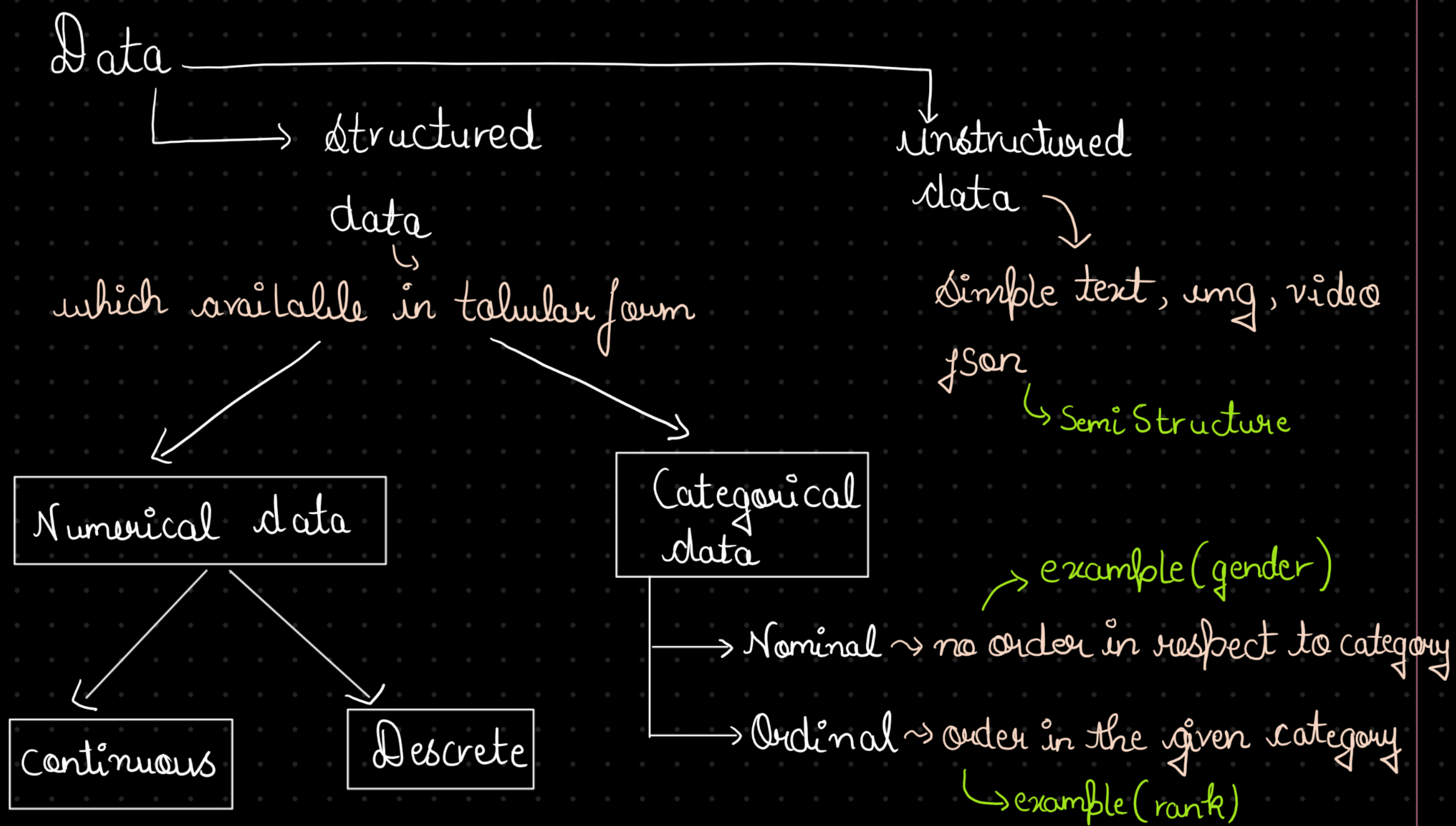
Descriptive Statistics

- ↳ summarizing our data with a single value

2) Inferential Statistics

- ↳ find out the conclusion for the population based on sample data

Example → There exist 10000 people in a city we take a sample of 3000 people to calculate avg height



Descriptive Statistics

1) mean

$\{1, 1, 2, 2, 3, 3, 4, 5, 5, 6\}$

$$\sum_{i=1}^n \frac{x_i}{N} ; \text{ where } N \text{ is total number of elements}$$

$\Rightarrow$

Statistics for Machine learning

divided into two parts:-

1) Descriptive Statistics

- ↳ Central tendency
- ↳ Dispersion method
- ↳ Probability theory
- ↳ Probability distribution Function

2) Inferential Statistics

- ↳ Sample/Population
- ↳ hypothesis testing
- ↳ Z test
- ↳ T test
- ↳ f test
- ↳ alpha test
- ↳ Chi Square test

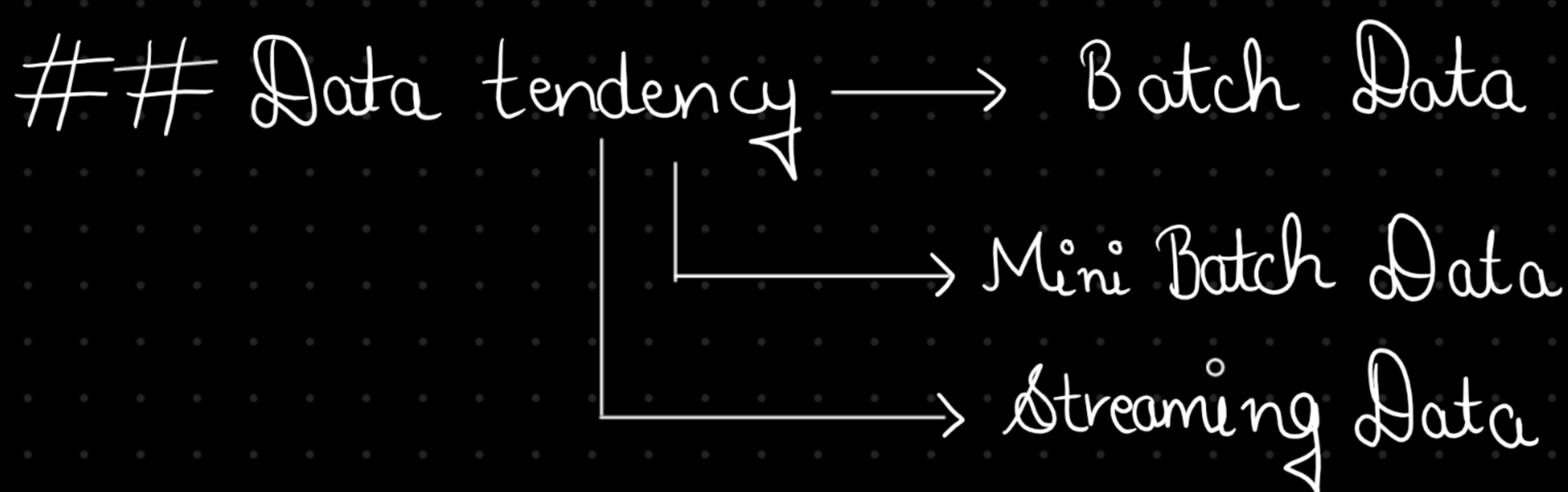
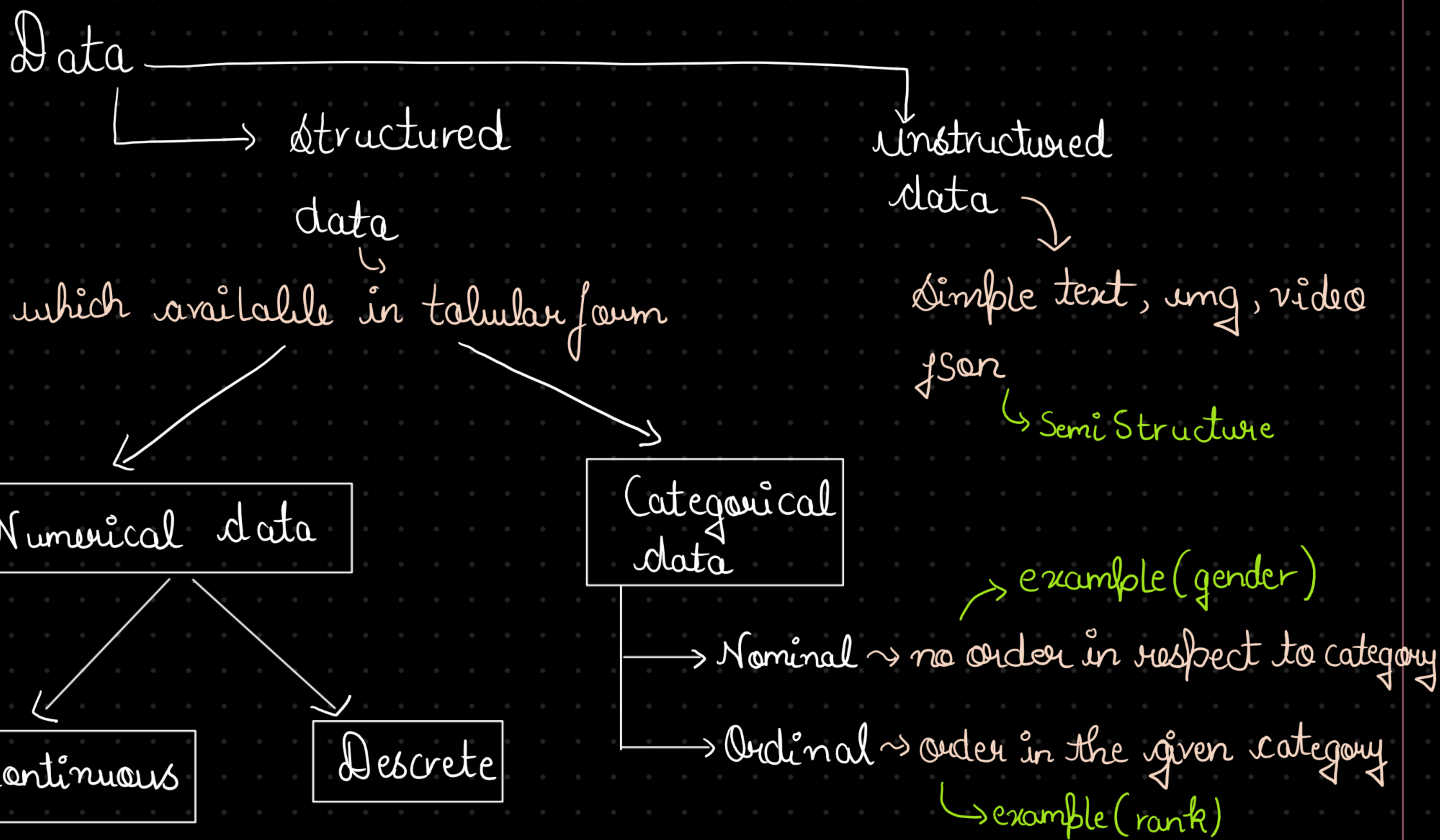
Descriptive Statistics

- ↳ summarizing our data with a single value

2) Inferential Statistics

- ↳ find out the conclusion for the population based on sample data

Example → There exist 10000 people in a city we take a sample of 3000 people to calculate avg height



Descriptive Statistics

1) mean

$\{1, 1, 2, 2, 3, 3, 4, 5, 5, 6\}$

$$\sum_{i=1}^n \frac{x_i}{N} ; \text{ where } N \text{ is total number of elements}$$

$$\Rightarrow 3.2$$

Notation for sample and population

measure	population parameter	sample statistics
mean	μ	$\bar{X}$
median	σ^2	S^2

mode

σ

S

2) Median

$$\begin{array}{ccc} \{1, 2, 3, 4, 5\} & & \{1, 2, 3, 4, 5, 6\} \\ \downarrow & \swarrow & \downarrow \\ 3 & & \frac{3+4}{2} = 3.5 \end{array}$$

values should be sorted!

3) Mode

$$\{1, 1, 1, 2, 2, 3, 3, 3, 3, 5, 5\}$$

most freq element $\leadsto 3$ [mode]

also should be sorted

Descriptive Data

1) Range $\leadsto$ maximum - minimum

$\leadsto$ sensitive to Outlier

2) Variance $\rightarrow$ Variance indicate how close or far the data points from the mean.

$$\text{Population Variance} \rightarrow \sigma^2 = \frac{1}{N} \cdot \sum_{i=1}^n (x_i - \mu)^2$$

$$\text{Sample Variance} \rightarrow s^2 = \frac{1}{n} \cdot \sum_{i=1}^n (x_i - \bar{x})^2$$

$\leadsto$ there is also formula for

with Bessel's correction were it

$$1/n-1.$$

3) Standard Deviation $\rightarrow \sqrt{\text{variance}}$

* General tips

1)

RL Education Tutor

Strategic Foundation for the RL tutor

↳ Core Vision [an AI tutor that adapts its teaching strategy to an individual student learning style, speed and current knowledge level]

So Based on this we have custom teaching materials for different type of student we can →

Student Activity → RL model adaptation → custom lesson delivery
continuous skill assessment.

Objective / Features Mapping →

1) Personalize lessons for each student: → The primary function is to move beyond static curriculum, this means the application dynamically select the next Content item (eg, a video, article) from possibly an larger pool. → ??? [how and where will extra pool of content], one way that comes to mind is example prep for IIT where upper batch were taught extra what but student which are struggling with standard syllabus???

2) Adjusting difficulty based on real time feedback: The system must respond to student performance. This will be implemented by including content of varying difficulty level within agent's action space. The agent's reward function will be engineered to incentivize the selection of appropriately challenging material content that is neither too easy to be unengaging nor too difficult to be discouraging.

3) Identify weak subjects early → Effective tutoring requires diagnosing and addressing area of weakness, This will be a vector that tracks

performance metrics across different topic or skills, These tracked metrics will serve as explicit input to the RL agent, allowing it to preferentially select content that targets and reinforces these weaker areas.

- Engagement with gamification: $\leadsto$ a user interface that provide immediate feedback through point, badges or progress bars.